

Amazon Reviews Sentiment Analysis

Introduction

The **Review Sentiment Analyzer** is a web-based application developed using Streamlit that helps users analyze product reviews. With an intuitive interface, the application enables users to select product categories, choose individual products, and analyze customer reviews using sentiment detection techniques. This tool aims to provide businesses and users with insight into public opinion on products by evaluating the proportion of positive, neutral, and negative sentiments expressed in user reviews.

Features

- **Review Sentiment Analysis by Product:** Select any product from a chosen category and analyze its review sentiments.
- **Visual Sentiment Charts:** View bar charts of positive and negative review distributions.
- **Download Reviews:** Export sentiment-scored reviews as a CSV file.
- **Category Analysis:** See all products in a category along with their positive review percentage and direct Flipkart links.
- **Product Comparison:** Compare sentiment metrics for two products side-by-side using separate positive and negative charts.

Sentiment detection logic

The core sentiment analysis is done using the **VADER (Valence Aware Dictionary and sEntiment Reasoner)** algorithm from the `nltk` library. Here's the logic breakdown:

- Reviews are split using `|` as a delimiter.
- Emojis and non-textual artifacts are stripped.
- Each sub-review is evaluated by VADER:
 - Compound score > 0.05 → **Positive**
 - Compound score < -0.05 → **Negative**
 - Else → **Neutral**

- Each product's final stats include:
 - % of Positive, Negative, and Neutral reviews
 - A detailed table of reviews with sentiment and score

Tech Stack

- **Frontend:** Streamlit
- **Backend:** Python
- **Web Scraping:** BeautifulSoup, `requests`
- **Sentiment Analysis:** VADER from `nltk`
- **Data Visualization:** `matplotlib`, `seaborn`, and Streamlit widgets

Implementation details

Web scraper:

The Review scraper is implemented using Python and automates the process of collecting product reviews from Flipkart/Amazon based on a specified category. It constructs the search URL dynamically, fetches the HTML content using the `requests` library, and parses it with `BeautifulSoup` to extract product links and names. For each product, it navigates to the detailed page, scrapes user reviews, and compiles them into a structured format. Reviews are cleaned and concatenated for clarity. The script then checks for an existing Excel file and appends the new data while removing duplicates. This ensures a growing, deduplicated dataset of product reviews that can be used for further analysis such as sentiment detection.

Sentiment Analysis:

The application follows a modular and interactive design:

1. **Data Cleaning:** Preprocessing to remove errors, empty reviews, and emojis.
2. **Sentiment Scoring:** Reviews processed through VADER and scored.
3. **Streamlit Interface:**
 - Sidebar with navigation and dropdown
 - Tabs for different views: Home, Analysis, Results, Compare

4. Charts & Tables:

- Dynamic Altair charts for review trends
- Tabular views of sentiment analysis

5. Download Feature: Ability to export filtered reviews in CSV format

Implementation code

scrapper.py

```
from bs4 import BeautifulSoup
import requests
import time
import pandas as pd
import os

category = "clocks"
url = f'https://www.flipkart.com/search?q={category}'
headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/122.0.0.0 Safari/537.36'
}

excel_file = "flipkartreviews1.xlsx"
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')

product_links = soup.find_all('a', class_='CGtC98', 'wjcEIp'])

results = []

for tag in product_links:
    href = tag.get('href')
    full_link = 'https://www.flipkart.com' + href if href else None
    name = None

    if 'CGtC98' in tag.get('class', []):
        name_tag = tag.find('div', class_='KzDlHZ')
        name = name_tag.text.strip() if name_tag else None
    elif 'wjcEIp' in tag.get('class', []):
        name = tag.get('title')

    if name and full_link:
        results.append({'name': name, 'link': full_link, 'category': category})
```

```

for product in results:
    try:
        response = requests.get(product['link'], headers=headers)
        soup = BeautifulSoup(response.text, 'html.parser')

        review_tags = soup.find_all('div', class_='ZmyHeo')

        reviews = []
        for tag in review_tags:
            review_text_div = tag.find('div')
            if review_text_div and review_text_div.text.strip():
                reviews.append(review_text_div.text.strip())

        product['review'] = " | ".join(reviews) if reviews else "No reviews found"

        print(f"Name: {product['name']}")
        print(f"Review: {product['review']}")
        print(f"Link: {product['link']}")
        print(f"Category: {product['category']}\n")

        time.sleep(1)

    except Exception as e:
        product['review'] = "Error"
        print(f"Error fetching review for {product['name']}: {e}")

new_data = pd.DataFrame(results, columns=["name", "review", "category", "link"])

if os.path.exists(excel_file):
    existing_data = pd.read_excel(excel_file)
    combined = pd.concat([existing_data, new_data],
ignore_index=True).drop_duplicates(subset=["name", "link"])
else:
    combined = new_data

combined.to_excel(excel_file, index=False)
print(f"Data written to '{excel_file}' successfully.")

```

main.py

```
import streamlit as st
```

```
import pandas as pd
import re
from nltk.sentiment.vader import SentimentIntensityAnalyzer
import nltk
import matplotlib.pyplot as plt
import altair as alt

nltk.download('vader_lexicon')

vader = SentimentIntensityAnalyzer()

category_map = {
    'acs': 'Air Conditioners',
    'air-purifiers': 'Air Purifiers',
    'beds': 'Beds',
    'blankets': 'Blankets',
    'blenders': 'Blenders',
    'cameras': 'Cameras',
    'chairs': 'Chairs',
    'chimneys': 'Kitchen Chimneys',
    'clocks': 'Clocks',
    'cookers': 'Pressure Cookers',
    'coolers': 'Air Coolers',
    'dishwashers': 'Dishwashers',
    'dolls': 'Dolls',
    'earphones': 'Earphones',
    'fans': 'Fans',
    'food-processors': 'Food Processors',
    'geysers': 'Water Heaters (Geysers)',
    'induction-stoves': 'Induction Stoves',
    'inverters': 'Inverters',
    'kettles': 'Electric Kettles',
    'keyboards': 'Keyboards',
    'laptops': 'Laptops',
    'lights': 'Lights',
    'mattresses': 'Mattresses',
    'memory-cards': 'Memory Cards',
    'microwaves': 'Microwave Ovens',
    'mixer': 'Mixer Grinders',
    'mobile-cables': 'Mobile Cables',
    'mobile-covers': 'Mobile Covers',
    'mobile-holders': 'Mobile Holders',
```

```

'monitors': 'Monitors',
'phone': 'Mobile Phones',
'power-banks': 'Power Banks',
'refridgerators': 'Refrigerators',
'rice-cookers': 'Rice Cookers',
'sewing-machines': 'Sewing Machines',
'sofas': 'Sofas',
'speakers': 'Speakers',
'tables': 'Tables',
'toasters': 'Toasters',
'toys': 'Toys',
'tv': 'Televisions',
'vacuum-cleaners': 'Vacuum Cleaners',
'washingmachines': 'Washing Machines',
'water-purifiers': 'Water Purifiers',
'wet-grinder': 'Wet Grinders',
}

```

```

def remove_emojis(text):
    if pd.isna(text):
        return text
    emoji_pattern = re.compile(
        "["
        "\U0001F600-\U0001F64F"
        "\U0001F300-\U0001F5FF"
        "\U0001F680-\U0001F6FF"
        "\U0001F1E0-\U0001F1FF"
        "\U00002500-\U00002BEF"
        "\U00002702-\U000027B0"
        "\U000024C2-\U0001F251"
        "]+", flags=re.UNICODE
    )
    return emoji_pattern.sub(r'', text)

```

```

def analyze_split_reviews(review):
    if pd.isna(review):
        return []

    parts = [part.strip() for part in review.split('|') if part.strip()]
    results = []

```

```

for part in parts:
    score = vader.polarity_scores(part)
    sentiment = (
        "positive" if score['compound'] > 0.05 else
        "negative" if score['compound'] < -0.05 else
        "neutral"
    )
    results.append({
        'review': part,
        'sentiment': sentiment,
        'score': round(score['compound'], 3)
    })

return results

@st.cache_data
def load_data():
    df = pd.read_excel('flipkartreviews1.xlsx', engine='openpyxl')
    df = df.drop_duplicates(subset='name', keep='first')
    df = df[df['review'] != "No reviews found"]
    df['review'] = df['review'].str.replace('READ MORE', '', regex=False)
    df['review'] = df['review'].apply(remove_emojis)
    df['category_clean'] = df['category'].map(category_map)
    return df

def main():
    st.set_page_config(page_title="Review Sentiment Analyzer", layout="wide")
    st.title("Review Sentiment Analyzer")

    menu = st.sidebar.radio("Navigation", ["Home", "Analyze Reviews", "Category
Analysis", "Compare Products"])
    df = load_data()

    if menu == "Home":
        st.markdown("### 🙌 Welcome to the Review Sentiment Analyzer")
        st.write("Use the sidebar to go to the **Analysis** section and explore product
reviews!")

    elif menu == "Analyze Reviews":
        st.header("🔍 Analyze Product Reviews")

```

```

categories = sorted(df['category_clean'].dropna().unique())
selected_category = st.selectbox("Choose a category", categories)

filtered_df = df[df['category_clean'] == selected_category]
product_names = sorted(filtered_df['name'].unique())

selected_product = st.selectbox("Choose a product", product_names)

if st.button("Review"):
    review_text = filtered_df[filtered_df['name'] ==
selected_product].iloc[0]['review']
    analysis = analyze_split_reviews(review_text)

    if analysis:
        result_df = pd.DataFrame(analysis)
        st.success(f"Showing {len(result_df)} reviews for
**{selected_product}**")
        st.dataframe(result_df, use_container_width=True)
        csv = result_df.to_csv(index=False).encode('utf-8')
        st.download_button(
            label="📁 Export Reviews as CSV",
            data=csv,
            file_name=f"{selected_product}_review_analysis.csv",
            mime='text/csv'
        )
    else:
        st.warning("No valid reviews to analyze.")

elif st.button("Show Results"):
    review_text = filtered_df[filtered_df['name'] ==
selected_product].iloc[0]['review']
    analysis = analyze_split_reviews(review_text)

    if analysis:
        result_df = pd.DataFrame(analysis)
        sentiment_counts =
result_df['sentiment'].value_counts().reindex(['positive', 'neutral', 'negative'],
fill_value=0)

        # Create figure with fixed size
        plt.figure(figsize=(5, 4))

```



```

        bars = plt.bar(sentiment_counts.index, sentiment_counts.values,
color=['green', 'gray', 'red'])
        plt.title(f"Sentiment Distribution for {selected_product}")
        plt.ylabel("Number of Reviews")
        plt.xlabel("Sentiment")
        plt.tight_layout()

        st.pyplot(plt.gcf()) # Use plt.gcf() to get the current figure

    else:
        st.warning("No valid reviews to visualize.")

elif menu == "Category Analysis":
    st.header("📊 Category-Level Sentiment Overview")

    categories = sorted(df['category_clean'].dropna().unique())
    selected_category = st.selectbox("Choose a category to analyze", categories)

    filtered_df = df[df['category_clean'] == selected_category]
    products = filtered_df['name'].unique()

    result_list = []

    for product in products:
        product_row = filtered_df[filtered_df['name'] == product].iloc[0]
        review_text = product_row['review']
        link = product_row['link']
        split_reviews = analyze_split_reviews(review_text)
        if split_reviews:
            total_reviews = len(split_reviews)
            positive_reviews = sum(1 for r in split_reviews if r['sentiment'] ==
'positive')
            positive_percent = round((positive_reviews / total_reviews) * 100, 2)

            result_list.append({
                'Product Name': product,
                'Positive Reviews (%)': positive_percent,
                'Total Reviews': total_reviews,
                'Flipkart Link': link
            })

    if result_list:

```

```

        result_df = pd.DataFrame(result_list).sort_values(by='Positive Reviews (%)', ascending=False)
        st.markdown("### Product Sentiment Summary")

        for i, row in result_df.iterrows():
            col1, col2, col3, col4 = st.columns([3, 2, 2, 2])
            with col1:
                st.write(f"***{row['Product Name']}***")
            with col2:
                st.write(f"{row['Positive Reviews (%)']}%")
            with col3:
                st.write(f"{row['Total Reviews']} reviews")
            with col4:
                st.link_button("Flipkart ", row["Flipkart Link"])

    elif menu == "Compare Products":
        st.title("📊 Compare Two Products")
        selected_category = st.selectbox("Select Category",
sorted(df['category'].unique()))
        category_products = df[df['category'] == selected_category]['name'].unique()
        col1, col2 = st.columns(2)
        with col1:
            product1 = st.selectbox("Select First Product", category_products,
key="product1")
        with col2:
            product2 = st.selectbox("Select Second Product", [p for p in
category_products if p != product1], key="product2")

        if st.button("Compare"):
            def get_sentiment_stats(product_name):
                row = df[(df['name'] == product_name) & (df['category'] ==
selected_category)].iloc[0]
                reviews = analyze_split_reviews(row['review'])
                total = len(reviews)
                positive = sum(1 for r in reviews if r['sentiment'] == 'positive')
                negative = sum(1 for r in reviews if r['sentiment'] == 'negative')
                pos_percent = round((positive / total) * 100, 2) if total else 0
                neg_percent = round((negative / total) * 100, 2) if total else 0
                return {
                    "Product": product_name,
                    "Positive %": pos_percent,

```

```

        "Negative %": neg_percent,
        "Total Reviews": total,
        "Link": row['link']
    }

stats1 = get_sentiment_stats(product1)
stats2 = get_sentiment_stats(product2)
sorted_stats = sorted([stats1, stats2], key=lambda x: x["Positive %"],
reverse=True)

st.markdown("### 📦 Product Summary")
col1, col2 = st.columns(2)
for stats, col in zip(sorted_stats, [col1, col2]):
    with col:
        st.markdown(f"#### {stats['Product']}")
        st.write(f"**Positive Reviews:** {stats['Positive %']}%")
        st.write(f"**Negative Reviews:** {stats['Negative %']}%")
        st.write(f"**Total Reviews:** {stats['Total Reviews']}")
        st.link_button("Flipkart 🔗", stats['Link'])

st.markdown("### 📊 Sentiment Comparison Charts")
chart_df = pd.DataFrame({
    "Product": [s["Product"] for s in sorted_stats],
    "Positive %": [s["Positive %"] for s in sorted_stats],
    "Negative %": [s["Negative %"] for s in sorted_stats],
})

st.markdown("#### ✅ Positive Reviews")
positive_chart = (
    alt.Chart(chart_df)
    .mark_bar(size=50, color="green")
    .encode(
        x=alt.X("Product:N", title=None),
        y=alt.Y("Positive %:Q", title="Percentage"),
        tooltip=["Product", "Positive %"]
    )
    .properties(height=300)
)

st.altair_chart(positive_chart, use_container_width=True)
st.markdown("#### ❌ Negative Reviews")
negative_chart = (
    alt.Chart(chart_df)
    .mark_bar(size=50, color="red")
    .encode(

```

```
        x=alt.X("Product:N", title=None),
        y=alt.Y("Negative %:Q", title="Percentage"),
        tooltip=["Product", "Negative %"]
    )
    .properties(height=300)
)
st.altair_chart(negative_chart, use_container_width=True)

if __name__ == "__main__":
    main()
```

Results

Home

<

Navigation

- Home
- Analyze Reviews
- Category Analysis
- Compare Products

Review Sentiment Analyzer

Welcome to the Review Sentiment Analyzer

Use the sidebar to go to the Analysis section and explore product reviews!

Deploy

Analyze reviews

- Navigation
- Home

Analyze Reviews

Category Analysis

Compare Products

Deploy

Review Sentiment Analyzer

Analyze Product Reviews

Choose a category

Air Conditioners

Choose a product

Blue Star 1 Ton 5 Star Split Inverter AC - White

Review

Show Results

- Navigation
- Home

Analyze Reviews

Category Analysis

Compare Products

Deploy

Review Sentiment Analyzer

Analyze Product Reviews

Choose a category

Dolls

Choose a product

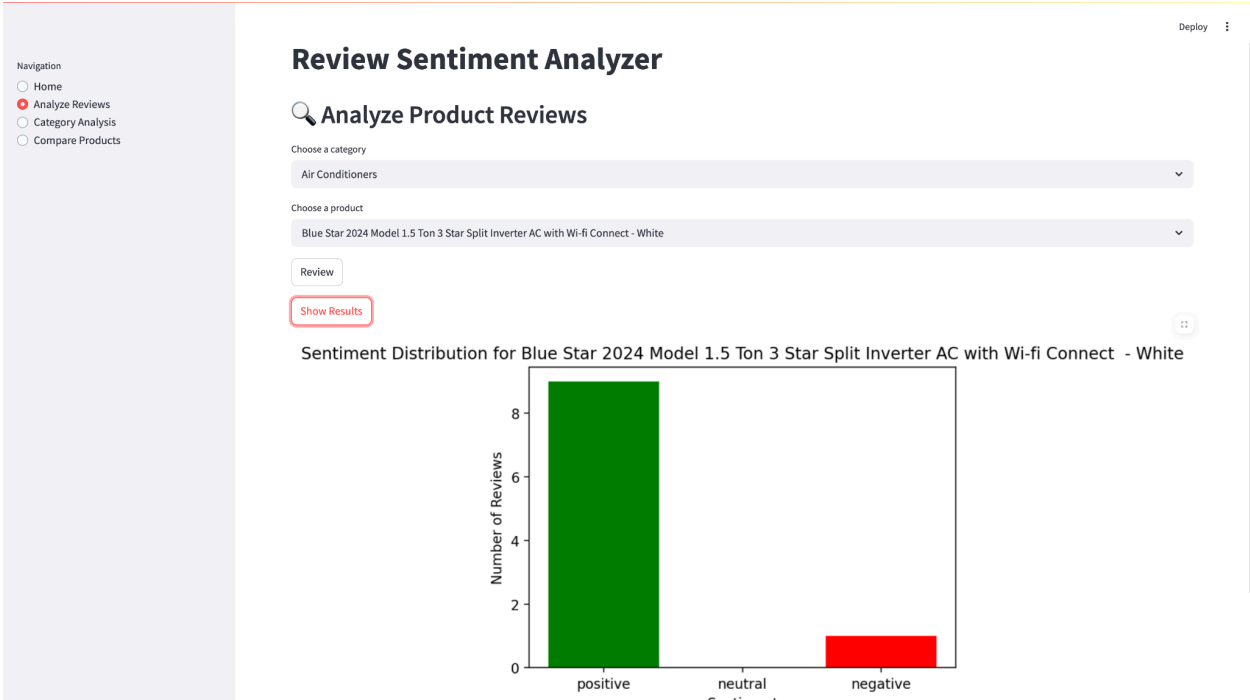
BARBIE Birthday Wishes Doll, Blonde in Pink Satin and Tulle Dress, Fashion Collectibles

Review

Showing 6 reviews for BARBIE Birthday Wishes Doll, Blonde in Pink Satin and Tulle Dress, Fashion Collectibles

	review	sentiment	score
0	Exactly as picture or i should say way more beautiful than the picture. Perfect quality.	positive	0.836
1	This is my childhood dream guys !! Finally i got my first Barbie	positive	0.379
2	Beautiful doll. My daughter liked it very much	positive	0.772
3	Beautiful,worth the money	neutral	0
4	My Sweetheart Barbie	positive	0.649
5	Loved it	positive	0.599

Export Reviews as CSV



Compare Products

Navigation

Home

Analyze Reviews

Category Analysis

Compare Products

Deploy

Review Sentiment Analyzer

Compare Two Products

Select Category

acs

Select First Product

Panasonic 2025 Model 1.5 Ton 3 Star Split Inverter AC with Wi-Fi Connect - White

Select Second Product

Haier 2025 Model Intelli-Convertible 7 in1 Convertible 1.5 Ton 3 Star Split Inverter Ex...

Compare

Product Summary

Haier 2025 Model Intelli-Convertible 7 in1 Convertible 1.5 Ton 3 Star Split Inverter Expandable HD Fil...

Positive Reviews: 90.0%

Negative Reviews: 0.0%

Total Reviews: 10

Flipkart

Panasonic 2025 Model 1.5 Ton 3 Star Split Inverter AC with Wi-fi Connect - White

Positive Reviews: 80.0%

Negative Reviews: 20.0%

Total Reviews: 10

Flipkart

Sentiment Comparison Charts

Positive Reviews

80

60

Conclusion

The Review Sentiment Analyzer is a robust, user-friendly tool for exploring and evaluating customer feedback on e-commerce products. By combining sentiment analysis, interactivity, and clear visual insights, the application serves both end-users and businesses seeking actionable intelligence from user reviews. Future expansions could include AI-driven summaries, and clustering reviews by topic for feature-specific analysis.